

Algoritmos Recursivos

Aula 02 - Comparação entre
algoritmos recursivos e iterativos



Funções e Procedimentos

Funções e Procedimentos

A linguagem C permite modularizar programas utilizando funções, facilitando a organização do código, a reutilização de código e a manutenção do software. As funções podem ser classificadas em procedimentos (sem retorno) e funções com retorno. Em C, as funções devem ser declaradas antes de serem usadas. A sintaxe básica de uma função é:

```
// Declaração de um procedimento  
void nome_do_procedimento (tipo parametro1, tipo parametro2, ...);
```

```
// Definição da função  
retorno nome_da_funcao (tipo parametro1, tipo parametro2, ...){  
    // Corpo da função  
    return valor; // Se houver retorno  
}
```

Procedimentos (Funções sem Retorno)

Os procedimentos são funções que não retornam valores e são declarados usando o tipo void.

```
#include <iostream>

void saudacao(){

    std::cout<<"Ola! Seja bem-vindo ao curso de C ! \n";

}

int main(){

    saudacao(); // Chamando o procedimento

    return 0;

}
```

Funções com Retorno

Uma função pode retornar um valor usando a palavra-chave return. O tipo de retorno deve ser especificado na assinatura da função.

```
#include <iostream>

int soma(int a, int b){
    return a + b;
}

int main(){
    int resultado = soma(5, 3);

    std::cout<<"A soma é: " << resultado << ". \n";

    return 0;
}
```

Funções Recursivas X Funções Iterativas

Funções Recursivas

A recursão é uma técnica de programação onde uma função chama a si mesma para resolver um problema. Essa abordagem é útil para problemas que podem ser divididos em subproblemas menores, como cálculos matemáticos, percursos em estruturas de dados e algoritmos de busca e ordenação.

Conceito de Recursão: uma função recursiva deve ter:

- **Caso base:** A condição de parada que impede chamadas infinitas.
- **Chamada recursiva:** A função chama a si mesma para resolver subproblemas menores.

A **estrutura** geral de uma função recursiva é:

```
retorno funcao (parametros){  
  
    if (condicao_base){  
  
        return resultado;  
  
    } else {  
  
        return funcao (novo_parametro);  
  
    }  
  
}
```

Fatorial de um Número

O fatorial de um número n é definido como: $n! = n \times (n - 1)!$ Com o caso base: $0! = 1$.

```
#include <iostream>

int fatorial(int n){
    if(n == 0){
        return 1; // Caso base
    }
    return n * fatorial(n - 1); // Chamada recursiva
}

// continua

int main(){
    int num = 5;

    std::cout << "Fatorial de " << num << " eh " <<
    fatorial(num) << ".\n";

    return 0;
}
```

Fatorial de um Número

O fatorial de um número n é definido como: $n! = n \times (n - 1)!$ Com o caso base: $0! = 1$.

```
#include <iostream>

int fatorialIterativo(int n){
    int resultado = 1;
    for (int i = 1; i <= n; i++){
        resultado *= i;
    }
    return resultado;
}

int main(){
    int num = 5;
    std::cout << "Fatorial de " << num << "eh: " << fatorialIterativo(num) <<
    ".\n";
    return 0;
}
```

// continua

Potência de um Número

Uma função recursiva para calcular XY .

```
#include <stdio.h>
```

```
int potencia(int x, int y){
```

```
    if (y == 0) return 1; // Caso base
```

```
    return x * potencia(x, y - 1);
```

```
}
```

```
int main(){
```

```
    int base = 2, expoente = 4;
```

```
    printf("%d^%d = %d\n", base, expoente, potencia(base, expoente));
```

```
    return 0;
```

```
}
```

Potência de um Número

```
#include <iostream>
```

```
int potencialiterativa(int x, int y){
```

```
    int resultado = 1;
```

```
    for(int i = 0; i < y; i++){
```

```
        resultado *= x;
```

```
    }
```

```
    return resultado;
```

```
} // continua
```

```
int main(){
```

```
    int base = 2, expoente = 4;
```

```
    std::cout << "Base: " << base << ", " << "expoente: " <<  
    expoente << ", " << "potencia: " << potencialiterativa(base,  
    expoente) << ".\n";
```

```
    return 0;
```

```
}
```

Sequência de Fibonacci

A sequência de Fibonacci é definida como: $F(n) = F(n - 1) + F(n - 2)$ Com os casos base: $F(0) = 0$ e $F(1) = 1$

```
#include <stdio.h>
```

```
int fibonacci(int n){
```

```
    if (n == 0) return 0;
```

```
    if (n == 1) return 1;
```

```
    return fibonacci(n - 1) + fibonacci(n - 2);
```

```
}
```

```
int main(){
```

```
    int num = 6;
```

```
    printf("O %dº termo de Fibonacci é %d\n",  
num, fibonacci(num));
```

```
    return 0;
```

```
}
```

Sequência de Fibonacci

```
#include <iostream>          int main(){
int fibonaccilterativo(int n){    int num = 6;
    if (n == 0) return 0;        std::cout << "0 " << num << "º termo de Fibonacci é : " << fibonaccilterativo(num) << ".\n";
    if (n == 1) return 1;        return 0;
    int a = 0, b = 1, c;        }
    for(int i = 2; i <= n; i++){
        c = a + b;
        a = b;
        b = c;
    }
    return b;
} //continua
```

Exercícios

Crie um programa que leia uma string (pode utilizar a biblioteca `<string.h>`) e apresente esta string invertida. Faça utilizando algoritmo recursivo e um outro programa utilizando algoritmo iterativo.

Crie um programa recursivo e outro iterativo que calcule o MDC (máximo divisor comum) de dois números.



C . E . S . A . R

Pessoas impulsionando inovação.
Inovação impulsionando negócios.

